

Core & Advanced Java

Complete Beginner Guide — 10 Page Edition

Examples · Diagrams · Interview Q&A

Java 8+

Collections

Concurrency

JVM

Infosys Ready

Table of Contents

- | | |
|--|--|
| 1. == vs equals() | 8. ArrayList vs LinkedList |
| 2. hashCode() & equals() Contract | 9. volatile & synchronized |
| 3. String / StringBuilder / StringBuffer | 10. Deadlock & Java Memory Model |
| 4. Immutability | 11. Heap / Stack / Metaspace |
| 5. Abstract Class vs Interface | 12. Garbage Collection |
| 6. Overloading vs Overriding | 13. LRU Cache Design |
| 7. HashMap Internals | 14. Cheat Sheet + Top 10 Interview Q&A |

How to read: Each page covers 1–2 topics with explanations, diagrams, code, and interview tips. Page 10 has the full cheat sheet.

1. == vs equals() · 2. hashCode() & equals() Contract

1. == vs equals()

`==` = same memory address (reference). `equals()` = same content/value (when overridden). For objects, always use `equals()` to compare values.

```
a → ["Hello"] (0x100)
b → ["Hello"] (0x200) ← different object
c → same as a
```

```
a==c → TRUE   a==b → FALSE
a.equals(b) → TRUE (same text)
```

Primitives: `==` compares values directly. **Integer trap:** `Integer` `a=100, b=100 → a==b` true (cached -128..127). `200==200` false (new objects).

```
String a=new String("Hi"), b=new String("Hi");
a==b;           // false – different objects
a.equals(b);    // true  – same content
int x=10,y=10; x==y; // true – primitives
```

Type	==	equals()
Objects	Same reference?	Same value?
Primitives	Compare values	N/A

Q: == vs equals()?

`==` compares addresses. `equals()` compares content. Always `equals()` for objects unless checking same reference.

2. hashCode() & equals() Contract

Rule 1: if `a.equals(b) → a.hashCode()==b.hashCode()` MUST hold

Rule 2: Same `hashCode` ≠ `equals()` true (collision OK)

Rule 3: Override BOTH together — always!

Analogy: `hashCode()` = shelf number in library. `equals()` = check exact book. Equal books MUST be on same shelf — else `HashMap` can't find them.

`HashMap` lookup:

1. `hashCode()` → bucket index

2. `equals()` → exact key in bucket

Wrong `hashCode` → wrong bucket → `get()` returns null!

```
class Employee {
    private int id;
    public boolean equals(Object o) {
        if(!(o instanceof Employee e)) return false;
        return id==e.id;
    }
    public int hashCode() {
        return Integer.hashCode(id); // SAME field!
    }
}
Set<Employee> s=new HashSet<>();
s.add(new Employee(101));
s.contains(new Employee(101)); // true
```

Mistakes: Override `equals` only · Different fields in `hashCode` · Mutate key after putting in `HashMap` · `HashSet` uses `HashMap` internally (keys = elements)

Q: Why hashCode with equals?

`HashMap` finds bucket via `hashCode`, exact match via `equals`. Breaking contract = silent data loss.

3. String / StringBuilder / StringBuffer · 4. Immutability

3. String vs StringBuilder vs StringBuffer

	String	StringBuilder	StringBuffer
Mutable?	NO	YES	YES
Thread-safe?	Yes (immutable)	No	Yes (sync)
Speed	Slow in loops	Fast	Slower
Use when	Constants, keys	String building (1 thread)	Multi-thread (rare)

Analogy: String = carved in stone (new stone for each change).
StringBuilder = whiteboard (erase & rewrite). StringBuffer = locked whiteboard.

```
String in loop (BAD):
s="" → s+"A" → NEW "A"
      → s+"B" → NEW "AB" (10K iter = 10K objects!)
```

```
StringBuilder (GOOD):
sb.append("A") → same buffer "A"
sb.append("B") → same buffer "AB"
```

```
// BAD
String s=""; for(int i=0;i<10000;i++) s=s+i;
// GOOD
StringBuilder sb=new StringBuilder();
for(int i=0;i<10000;i++) sb.append(i);
String r=sb.toString();
```

Tip: String pool — literals "Hi" may be cached. `new String("Hi")` always creates heap object.

4. Immutability

Object **cannot change** after creation. Any "change" creates a **new** object. String, Integer, LocalDate are immutable.

```
String name = "Ashish"
name → [A|s|h|i|s|h] (fixed)

name.concat(" Shah")
→ NEW object [Ashish Shah]
→ name still "Ashish"!
```

Benefits: Thread-safe · Safe HashMap keys (hash never changes) · Predictable · Secure for passwords · Can share without copying

How to Make Immutable Class

1. `final` class (no subclass)
2. `private final` fields
3. No setters
4. Defensive copy for mutable fields (List, Date)

```
public final class Person {
    private final String name;
    private final List<String> tags;
    public Person(String n, List<String> t) {
        name=n; tags=List.copyOf(t);
    }
    public String getName(){return name;}
}
// Java 17 Record – immutable by default
public record User(Long id, String email){}
```

Interview: Why String is good HashMap key? hashCode never changes because object never mutates.

5. Abstract Class vs Interface · 6. Overloading vs Overriding

5. Abstract Class vs Interface

	Abstract Class	Interface
What	Incomplete class	Contract / capability
Inherit	extends (ONE only)	implements (MANY)
Fields	Any instance fields	static final constants
Constructor	Yes	No
Methods	Abstract + concrete	Abstract + default/static (Java 8+)
Use when	Shared code + "is-a"	"can do" role (Flyable)

Analogy: Abstract class = partially built house (inherit finished rooms). Interface = job description (can have multiple skills).

```
Animal (abstract)
  abstract sound(); breathe(){...}
  /           \
Dog             Cat

Flyable   Swimmable (interfaces)
  \       /
  Duck implements both
```

```
abstract class Animal {
    protected String name;
    Animal(String n){name=n;}
    abstract void sound();
    void sleep(){System.out.println(name+" sleep");}
}
interface Flyable { void fly(); }
class Bird extends Animal implements Flyable {
    void sound(){System.out.println("Chirp");}
    public void fly(){System.out.println("Flying");}
}
```

6. Overloading vs Overriding

	Overloading	Overriding
Name	Compile-time poly.	Runtime poly.
Where	Same class, diff params	Child, same signature
Return type	Can differ	Same or covariant
Decided by	Compiler	JVM at runtime

OVERLOADING (same class):
print(int) print(String) print(int,int)
→ compiler picks by argument type

OVERRIDING (parent → child):
Animal a = new Dog();
a.speak(); → "Bark" (JVM uses Dog's version)

```
// OVERLOADING
class Calc {
    int add(int a,int b){return a+b;}
    double add(double a,double b){return a+b;}
}
// OVERRIDING
class Animal{ void speak(){System.out.println("Sound");} }
class Dog extends Animal{
    @Override void speak(){System.out.println("Bark");}
}
Animal a=new Dog(); a.speak(); // Bark
```

Tip: Always use @Override when overriding — compile error if parent method missing.

Q: Overload vs Override?

Overload = same name, different params, same class, compile-time.
Override = child replaces parent method, runtime decision.

7. HashMap Internals — Complete Breakdown

Simple: Array of buckets. Key → hashCode() → bucket index → store/lookup. Like numbered mailboxes in a building. Average O(1) lookup.

```
Node<K,V>[] table (default size 16, power of 2)

[0]→null [1]→(Banana→v) [2]→null
[3]→(Apple→v)→(Apricot→v) ← collision: linked list
...
[15]

List >8 in bucket + table≥64 → Red-Black Tree
Load factor 0.75 → size > cap×0.75 → DOUBLE + rehash
```

put(key, value) Steps

1. hash = spread(key.hashCode())
2. index = (table.length-1) & hash
3. Bucket empty → insert Node
4. Collision → linked list/tree, check equals()
5. Over threshold → resize ×2, rehash all

get(key) Steps

1. Compute hash → bucket index
2. Traverse bucket, compare equals()
3. Return value or null

Property	Detail
Thread-safe?	NO — use ConcurrentHashMap
Null key	One null key allowed
Null values	Multiple OK
Key rule	Immutable + correct equals/hashCode
Avg complexity	O(1) get/put
Worst case	O(log n) with trees
Default capacity	16, load factor 0.75
HashSet	Uses HashMap — elements are keys

```
Map<String,Integer> map=new HashMap<>();
map.put("Apple",80);
map.put("Banana",40);
map.get("Apple"); // hash→bucket→equals→80

// Thread-safe:
Map<String,Integer> safe=
    new ConcurrentHashMap<>();
```

Interview: Not thread-safe. Keys must be immutable. Collision = linked list then tree. Why power of 2? Fast index via bitwise AND.

Q: HashMap internal working?

Array of buckets. hashCode()→index. Collision→linked list→tree if >8. Load 0.75→resize×2. get() uses hash+equals.

8. ArrayList vs LinkedList · 9. volatile · 10. synchronized

8. ArrayList vs LinkedList

Op	ArrayList	LinkedList
Structure	Dynamic array	Doubly linked nodes
get(i)	O(1) fast	O(n) slow
add end	O(1)	O(1)
add middle	O(n) shift	O(1)*
Memory	Less	More (pointers)
Use	95% of cases	Rare

ArrayList: [10|20|30|40|__]
get(2)→30 instantly

LinkedList: null←[10]↔[20]↔[30]→null
get(2)→walk 3 nodes

Rule: Use ArrayList unless you need Deque/Queue behavior.

9. volatile

Fixes **visibility** — all threads see latest value.
Does NOT make count++ atomic.

WITHOUT: T1 writes to CPU cache, T2 reads stale value
WITH: T1 writes→main memory, T2 reads latest

```
volatile boolean shutdown=false;
// Good for flags
```

```
volatile int count=0;
count++; // NOT SAFE!
// Use AtomicInteger
```

10. synchronized

Only **one thread** at a time. Mutual exclusion + visibility.

balance=1000
T1: sync withdraw(600)→LOCK→400
T2: sync withdraw(600)→WAIT→reads 400→safe

```
class Counter {
    int count=0;
    synchronized void inc(){count++;}
}
synchronized(lockObj){
    // critical section
}
```

	volatile	synchronized	AtomicInteger
Visibility	Yes	Yes	Yes
count++ safe	No	Yes	Yes
Blocks threads	No	Yes	No (CAS)
Best for	Flags, on/off	Critical sections	Counters, metrics

Q: volatile vs synchronized?

volatile = visibility only. synchronized = visibility + mutual exclusion (safe i++). AtomicInteger = lock-free atomic counter.

11. Deadlock · 12. Java Memory Model (JMM)

11. Deadlock

Threads wait for each other **forever**. Each holds a lock the other needs. Nobody proceeds.

```
Thread 1          Thread 2
lock A ✓          lock B ✓
wants B → WAIT    wants A → WAIT
----- DEADLOCK -----
```

4 Coffman Conditions (ALL required):
1. Mutual exclusion 2. Hold-and-wait
3. No preemption 4. Circular wait

```
Object lockA=new Object(), lockB=new Object();
// Thread 1
synchronized(lockA){ synchronized(lockB){} }
// Thread 2 - OPPOSITE ORDER = DEADLOCK!
synchronized(lockB){ synchronized(lockA){} }
```

Prevention

1. **Lock ordering** — always A then B everywhere
2. **tryLock(timeout)** — don't wait forever
3. **Minimize lock scope**
4. **Detect:** jstack → "Found Java-level deadlock"

Q: What is deadlock?

Two threads circular-wait for locks. Fix: consistent lock order, tryLock with timeout.

12. Java Memory Model (JMM)

Rules for how threads see memory — visibility, ordering, atomicity across CPU cores.

3 Problems JMM Solves

- **Visibility** — T1 sets x=1, T2 still sees 0 (CPU cache)
- **Ordering** — CPU reorders instructions; T2 sees partial object
- **Atomicity** — count++ = read+add+write; two threads lose increment

Tool	Visibility	Atomicity
volatile	Yes	No (for i++)
synchronized	Yes	Yes (in block)
AtomicInteger	Yes	Yes (CAS)
final fields	Yes (post-construct)	N/A

Happens-Before Rules

- Unlock → subsequent Lock (same monitor)
- volatile write → subsequent volatile read
- thread.start() → actions in new thread
- Thread actions → thread.join() returns

13. Heap / Stack / Metaspace · 14. Garbage Collection

13. JVM Memory Areas

JVM MEMORY	
STACK (per thread)	HEAP (GC)
- method frames	- new()
- local variables	- arrays
- object references	- strings
auto-freed on	freed when
method return	no refs
METASPACE (Java 8+, not PermGen)	
- class definitions, bytecode	

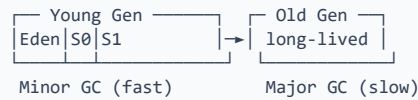
```
static int x=10;    // Metaspace
void method() {
  int local=5;      // STACK
  String s="Hi";    // ref→STACK, "Hi"→HEAP
  Order o=new Order();// ref→STACK, obj→HEAP
} // stack frame gone; heap waits for GC
```

Area	Stores	GC?
Stack	Locals, method frames	No
Heap	All objects (new)	Yes
Metaspace	Class metadata	Rarely

14. Garbage Collection

Auto-frees memory for objects with **no references**. Never call delete in Java.

HEAP:



Lifecycle: new()→Eden → survive GC→Survivor
→ more GC→Old Gen → no refs→collected

```
Employee e=new Employee();
e=null; // eligible for GC
```

```
Employee e1=new Employee();
Employee e2=e1;
e1=null; // still reachable via e2
```

GC Type	Cleans	Speed
Minor GC	Young gen	Fast (ms)
Major GC	Old gen	Slow
Full GC	Entire heap	Slowest
G1 (Java 9+)	Regions	Balanced default

Production: CPU spike? Run jstack + check GC logs before restart.

15. LRU Cache Design

LRU = Least Recently Used. Fixed capacity cache. When full, evict the item not accessed longest. Like a small desk — remove the paper you haven't looked at in the longest time.

Operations (Capacity = 3)

```
put(A) → [A]
put(B) → [A,B]
put(C) → [A,B,C]    ← full
get(A) → [B,C,A]    ← A = most recent
put(D) → [C,A,D]    ← B evicted (least recent)
```

MRU (head) → LRU (tail/evict)

Approach 1: LinkedHashMap

```
class LRUCache<K,V> extends LinkedHashMap<K,V> {
    private final int cap;
    LRUCache(int cap) {
        super(cap, 0.75f, true); // access-order
        this.cap = cap;
    }
    protected boolean removeEldestEntry(
        Map.Entry<K,V> e) {
        return size() > cap;
    }
}
// get(1) then put(4,D) evicts key 2
```

Approach 2: HashMap + Doubly Linked List (Interview)

HashMap: key → Node (O(1) lookup)
DLL: HEAD(MRU) ↔ nodes ↔ TAIL(LRU)

get(key): map lookup → move to HEAD O(1)
put(key): exists → update, move HEAD O(1)
 new+full → remove TAIL, add HEAD O(1)
 new+space → add at HEAD O(1)

Total: get O(1), put O(1), Space O(cap)

```
void get(int key) {
    if(!map.containsKey(key)) return -1;
    Node n=map.get(key);
    moveToHead(n);
    return n.val;
}
void put(int key,int val) {
    if(map.containsKey(key)) updateMoveToHead(key,val);
    else {
        if(map.size()==cap) removeTail();
        addToHead(key,val);
    }
}
```

Q: Design LRU Cache O(1)?

HashMap for O(1) lookup + DLL for O(1) move/evict. HEAD=MRU, TAIL=LRU.

16. Cheat Sheet + Top 10 Infosys Interview Q&A

#	Topic	One-Line Answer	Must-Know
1	== vs equals	== address; equals content	equals() for objects
2	hashCode+equals	Equal → same hashCode	Override both; same fields
3	String/Builder/Buffer	Immutable/fast mutable/sync	StringBuilder in loops
4	Immutability	Can't change after creation	final class+fields, no setters
5	Abstract vs Interface	1 inheritance vs many contracts	Interface=can do; Abstract=is-a
6	Overload vs Override	Same class/diff params vs child replaces	Compile vs runtime
7	HashMap	Buckets+hash+equals	Not thread-safe; load 0.75
8	ArrayList vs LinkedList	Array=fast read; Linked=fast insert	Use ArrayList 95%
9	volatile	Visibility, not atomic	Flags yes; count++ no
10	synchronized	One thread at a time	Small lock scope
11	Deadlock	Circular lock wait	Consistent lock order
12	JMM	Thread memory rules	happens-before
13	Stack/Heap/Meta	Locals/objects/class data	new()→heap
14	GC	Auto-delete unreferenced	Young=minor; Old=major
15	LRU Cache	Evict least recent	HashMap+DLL=O(1)

Top 10 Interview Q&A

Q1. HashMap internals? Bucket array. hashCode()→index. Collision→list→tree if >8. Load 0.75→resize×2.

Q2. == vs equals? == = reference. equals = content. Use equals for objects.

Q3. String vs StringBuilder? String immutable (new object each concat). StringBuilder mutable, fast.

Q4. Abstract vs Interface? Abstract: 1 inheritance, shared code. Interface: multiple, capability.

Q5. Overload vs Override? Overload: same class, diff params, compile-time. Override: child replaces, runtime.

Q6. Deadlock? Circular lock wait. Fix: consistent order, tryLock timeout.

Q7. volatile vs synchronized? volatile=visibility. synchronized=visibility+mutual exclusion.

Q8. ArrayList vs LinkedList? ArrayList O(1) get. LinkedList O(n) get. Use ArrayList.

Q9. How GC works? No references→eligible. Young=minor GC. Old=major GC. G1=Java 9+ default.

Q10. Design LRU Cache? HashMap O(1) lookup + DLL O(1) move/evict. Remove tail when full.

Print: A4, enable Background graphics. **Interview:** Explain with project examples. Say "I" not "we".